

category: avalonia

tags: [config-tool, crypto, unhandled-exception, chicken-egg-bug, sql-config]

severity: high

created: 2026-07-24

updated: 2026-07-24

project-origin: iPGSv4 (ApplicationConfig)

App config tool tự crash khi mở, không sửa được config đã hỏng (chicken-egg bug)

Tình huống gặp phải

Đang làm gì? Tính năng gì? Môi trường nào?

Project `ApplicationConfig` (Avalonia, tool cấu hình SQL/LPR/Parking cho iPGSv4) — user báo: "khi chưa có config, config sai không kết nối được, không mở được app config => cần sửa lại". Tức là khi `sql.txt` bị sai/hỏng, mở app config lên để sửa lại thì app config crash luôn — không có cách nào vào để sửa.

Triệu chứng / Lỗi

Không có exception log rõ ràng phía user (WinExe không có console) — chỉ biết "không mở được app config". Khi trace code: `SqlConfigView.OnLoaded` gọi `SQLConfig.GetDecodePassword(config.Password)` không có try/catch → nếu `config.Password` không phải Base64 hợp lệ (config bị sửa tay sai, hoặc field cũ/khác định dạng), `Convert.FromBase64String` trong `CryptorEngine.Decrypt` ném `FormatException` ngay trong `OnLoaded` của `UserControl`. Avalonia không tự bọc exception trong sự kiện `Loaded` trên desktop lifetime → toàn bộ process crash khi `MainWindow.OnLoaded` set `_sqlView` vào `TabItem.Content` (kích hoạt `SqlConfigView.OnLoaded` đồng bộ).

Nguyên nhân gốc rễ (Root Cause)

- `NewtonSoftHelper<T>.DeserializeObjectFromPath` đã null-safe (file không tồn tại / rỗng / JSON lỗi → trả `null`, không throw) — nên case "chưa có config" tự nó KHÔNG crash.
- Nhưng case "config sai" (file tồn tại, JSON hợp lệ, nhưng field `Password` không phải chuỗi Base64 decrypt được — do sửa tay, do format cũ, do version encrypt khác) thì `SQLConfig.GetDecodePassword()` throw, và lời gọi đó nằm ngoài mọi try/catch trong `SqlConfigView.axaml.cs`.
- So sánh: `LoadingWindow.axaml.cs` (app chính IPGSUseCam) gọi cùng `GetDecodePassword` nhưng đã bọc trong try/catch tổng ở vòng lặp load task → app chính chỉ báo lỗi kết nối, không crash. `ApplicationConfig` (tool sửa config) lại thiếu lớp bảo vệ tương tự — ngược đời vì đây chính là tool user cần dùng để SỬA config khi nó sai.

Giải pháp

```
// ApplicationConfig/Views/SqlConfigView.axaml.cs - trong OnLoaded, sau khi gán
_serverName/_username
try
{
    _password.Text = SQLConfig.GetDecodePassword(config.Password);
}
catch (Exception)
{
    _password.Text = "";
    ShowStatus("⚠ Mật khẩu đã lưu bị lỗi định dạng - vui lòng nhập lại", success:
false);
}
```

1. Bọc riêng lẻ đúng đoạn `GetDecodePassword` (không bọc cả block `if (config is not null)`) để các field khác (`ServerName`, `Authentication`, `DatabaseName`) vẫn load bình thường dù `Password` hỏng.
2. Fallback về chuỗi rỗng, không để `_password.Text` null hay giữ giá trị cũ gây hiểu nhầm.
3. Hiển thị status cảnh báo ngay trên UI (dùng `ShowStatus` sẵn có của view) để user biết cần nhập lại password, không chỉ âm thầm bỏ qua.

Áp dụng lại (How to reuse)

- Khi thấy 1 tool có nhiệm vụ "sửa config" mà lại đọc/parse chính config đó khi khởi động (không phải chỉ ghi) → LUÔN kiểm tra: mọi bước decode/decrypt/parse dữ liệu từ config cũ có được bọc try/catch riêng không? Đây là điểm crash kinh điển kiểu "con gà quả trứng" — config hỏng thì không mở được tool để sửa config.
- Bất kỳ chỗ nào gọi `CryptorEngine.Decrypt` / `SQLConfig.GetDecodePassword` với dữ liệu đọc từ file (không phải vừa mới encrypt trong cùng phiên) → PHẢI coi là input không tin cậy, bọc try/catch.
- Kiểm tra `NewtonSoftHelper<T>.DeserializeObjectFromPath` null-safe không đồng nghĩa toàn bộ pipeline load config an toàn — vẫn phải soát các bước xử lý field bên trong object trả về (đặc biệt field đã encrypt).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Đừng bọc try/catch bao trùm cả `if (config is not null) { ... }` — sẽ vô tình bỏ qua luôn `Authentication/DatabaseName` nếu chỉ `Password` lỗi, làm user mất thêm dữ liệu đã có.
- ⚠ `Convert.FromBase64String("")` KHÔNG throw (trả mảng rỗng) — bug này chỉ xảy ra khi `Password` có nội dung nhưng không phải Base64 hợp lệ hoặc sai độ dài khối TripleDES, không phải khi `Password` rỗng.
- ⚠ Avalonia Desktop lifetime không có global handler mặc định cho exception trong `OnLoaded` của control — 1 exception nhỏ ở 1 tab con có thể kill toàn bộ app. Cần nhắc thêm `AppDomain.CurrentDomain.UnhandledException` ở `Program.cs` cho các tool nhỏ dạng config editor để tránh lặp lại pattern này ở view khác trong tương lai.

Tham chiếu

- File: `ApplicationConfig/Views/SqlConfigView.axaml.cs`

- File liên quan (đã có try/catch đúng cách, dùng làm tham chiếu):
[IPGSUseCam/Views/LoadingWindow.axaml.cs](#)
- Project liên quan: iPGSv4 (nhánh zcu-avalonia)